T07 Pathfinding (optional)

CRE132 worksheet

What you'll make

In T06 and T06b the slime behind the pond ran straight at you and got stuck at the water's edge. Here you'll give the slimes **pathfinding**: they plan a route round water and fences and follow it. The slime in the fenced pen heads for the gate, comes out and chases you round the outside; the slime behind the pond runs round the end of it.

You won't write a new brain. You'll copy T06b's `EnemyAI` and change only the movement, the slime's "legs". By the end you'll know what a **NavMesh** is, how to bake one for a 2D tilemap scene, and how a **NavMeshAgent** follows it.



Figure 1: The finished meadow: the player on a patch of dirt, with the pen's fence above and a row of trees below.

Before you start

1. Unzip `CRE132_T07_Pathfinding_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T07_Pathfinding_Starter` folder. Open it with Unity **6000.3.11f1**. The first time, Unity also downloads the NavMeshPlus package, which can take a little longer.
2. Open the scene `Assets/_game/Scenes/TopDown/T07_Pathfinding`.
3. Press **Play**. You can walk and swing the sword, but the slimes don't move. Press Play again to stop.

The scene is T06's meadow with a fenced pen (**Slime 2** lives inside, and the gate is at the bottom) and a long pond (**Slime 1** lives behind it). The slimes have no brain yet, and there's no NavMesh: no **NavMesh** object, nothing marked as walkable, and no **Nav Mesh Agent** on the slimes. Parts 1 to 4 set that up, the way you would in your own game; part 5 is the script.

You'll also need your `EnemyAI.cs` from T06b, with the Attack and ReturnHome states. If you don't have it, use the one in `CRE132_T06b_EnemyAttack_Finished`, in `Assets/_game/Scripts/T06_EnemyAI`.

1. The NavMeshPlus package

A **NavMesh** is a map of where a character can stand. Unity's own NavMesh only understands 3D floors lying flat. **NavMeshPlus** is a free package that adds 2D: it can build a NavMesh from Tilemaps, sprites and 2D colliders.

This project already has it: open **Window > Package Manager** and you'll find **NavMeshPlus** under **In Project**. To add it to your own project, click **+** at the top left of the Package Manager, choose **Install package from git URL** and paste:

```
https://github.com/h8man/NavMeshPlus.git
```

(This project pins one exact version in `Packages/manifest.json`, so it can't change under us.)

2. The surface

1. Right-click an empty part of the Hierarchy and choose **Create Empty**. Name it `NavMesh`.
2. With **NavMesh** selected, click **Add Component** in the Inspector, type `Navigation Surface` and pick it. (It's also under **Navigation > Navigation Surface**.)
3. Click **Add Component** again and add **Navigation CollectSources2d**. Tick its **Override By Grid**.
4. Click **Rotate Surface to XY** on CollectSources2d. The object's **Rotation X** becomes **−90**, and the yellow warning under the button goes away.
5. On the Navigation Surface, open **Advanced** and tick **Hide Editor Logs**, so baking doesn't fill the Console with messages. Leave everything else as it is.

- **Navigation Surface** builds and holds the NavMesh.
- **Navigation CollectSources2d** collects the 2D things to build it from. **Override By Grid** makes it use the scene's **Grid** and its tilemaps, with every tile counted as a full square whatever the shape of its sprite.
- **The rotation:** the NavMesh system works on a flat floor, and **Rotate Surface to XY** turns its floor to face our camera.

Test it: select **NavMesh** and check its **Transform** shows **Rotation** −90, 0, 0, with both components under it. There's nothing to bake yet: the next part says what to bake.

3. Saying what's walkable

Only objects with a **Navigation Modifier** are baked into the NavMesh. Open **Grid** in the Hierarchy and set up its tilemaps:

1. Select **Ground** and add a **Navigation Modifier** (**Add Component > Navigation > Navigation Modifier**). Leave it as it is: with nothing overridden, the ground is walkable.
2. Select **Water**, add a **Navigation Modifier**, tick **Override Area** and set **Area** to **Not Walkable**.
3. Do the same on **Fences**: **Override Area** ticked, **Area Not Walkable**.

Test it: select each of the three tilemaps in turn and check its Navigation Modifier. Only **Water** and **Fences** have **Not Walkable**.

4. Baking and the agent

Select the **NavMesh** object and click **Bake** at the bottom of the Navigation Surface. It saves the result as `NavMesh-NavMesh.asset`, in a new folder next to the scene: `Scenes/TopDown/T07_Pathfinding`.

With NavMesh selected, the Scene view shows the result in blue: holes round the water and the fences, with a gap through the pen's gate. The blue stops a little short of each obstacle, by the agent's radius, so a slime's middle never gets too close.

The NavMesh is baked in the editor, not while playing. If you paint more water or fences, bake again.

Now the slimes. Select **Slime 1** and Ctrl-click **Slime 2** (Cmd-click on a Mac) so both are selected:

1. Click **Add Component > Navigation > Nav Mesh Agent**.
2. On the **Nav Mesh Agent**, set **Radius** to **0.4** and **Acceleration** to **20**. Check **Stopping Distance** and **Base Offset** are **0**. Leave the rest.
3. On the **Rigidbody 2D**, set **Body Type** to **Kinematic**.

- **The agent plans a path across the NavMesh and moves the slime along it.** Its **Speed** doesn't matter: the script sets it.
- **The Rigidbody 2D is Kinematic** because the agent moves the slime, not physics. A Dynamic body would fight the agent and could be pushed off its path.

Test it: select **NavMesh** again and look at the blue in the Scene view: both slimes stand on it, Slime 2 inside the pen with a way out through the gate. Don't press Play yet: until the script in part 5 tells the agent it's in 2D, the agent turns the slimes edge-on to the camera.

5. PathfindingEnemyAI: new legs, same brain

Make a folder `Assets/_game/Scripts/T07_Pathfinding` and create a script `PathfindingEnemyAI` in it. Open your T06b `EnemyAI.cs` in your code editor, select everything, copy it, and paste it over everything in `PathfindingEnemyAI`. Then change the class line to:

```
public class PathfindingEnemyAI : MonoBehaviour
```

The class name must match the file name. Now make these changes; everything else stays exactly as it was.

At the top, replace the comment and add the second `using` line (`UnityEngine.AI` is where `NavMeshAgent` lives):

```
// T06's enemy state machine, moving with a NavMeshAgent so it finds its way round water
// and fences. Only the movement changed: the states and their rules are the same as T06's
// EnemyAI. The agent needs a baked NavMesh (a NavMeshSurface from the NavMeshPlus
// package, rotated to face the 2D camera). The states:
//   Idle       wait a moment, then Wander (or Chase if the player comes close)
//   Wander     walk to a random spot near home, then Idle (or Chase if the player comes
//              close)
//   Chase      run at the player; Attack when close enough, ReturnHome if the player gets
//              away
//   Attack     jump on the player, then go back to Chase
//   ReturnHome walk back home, then Idle
using UnityEngine;
using UnityEngine.AI;
```

The fields: change `private Rigidbody2D body;` to:

```
    private NavMeshAgent agent;
```

and delete these lines, because the agent now does the moving:

```
    // Set by the states each frame; FixedUpdate turns them into a velocity.
    private Vector2 moveDirection;
    private float moveSpeed;
```

`Awake`: replace the `body = GetComponent<Rigidbody2D>();` line, so it starts like this:

```
    private void Awake()
    {
        agent = GetComponent<NavMeshAgent>();
        // The NavMeshAgent was made for 3D, where "up" is the Y axis. Stop it turning the
        // slime to stand up out of the screen, and from rotating it to face where it
        // walks.
        agent.updateRotation = false;
        agent.updateUpAxis = false;
        // The Animator is on the Sprite child (see T03).
        animator = GetComponentInChildren<Animator>();
```

- **`updateRotation` and `updateUpAxis` off**: the NavMeshAgent was made for 3D, where “up” is the Y axis. Without these two lines, it would tip the slime up out of the screen and turn it to face where it walks.

Delete the whole `FixedUpdate` method. Nothing sets a velocity any more.

`ChangeState`: change `moveDirection = Vector2.zero;` to `StopMoving();`, so the method starts like this:

```
    // Switches state. Everything that should happen once, as a state starts, goes here.
    private void ChangeState(State newState)
    {
        Debug.Log(gameObject.name + ": " + currentState + " -> " + newState);
        currentState = newState;
        stateTimer = 0f;
        StopMoving();
```

Then, in its `switch`, change each `moveSpeed =` to `agent.speed =`. There are three: in **Wander**, **Chase** and **ReturnHome**. For example:

```
                agent.speed = chaseSpeed;
```

`MoveTowards`: replace the method with this one, and add `StopMoving` straight after it:

```
    // The agent plans a path round anything Not Walkable and follows it. Setting the
    // destination every frame keeps the path up to date while the player moves; for a few
    // enemies that's cheap. A spot off the NavMesh (in the pond) sends it to the nearest
    // point it can reach.
    private void MoveTowards(Vector2 target)
    {
        agent.isStopped = false;
        agent.SetDestination(target);
    }

    private void StopMoving()
    {
        agent.isStopped = true;
        agent.ResetPath();
    }
```

- **`SetDestination`** asks the agent to plan a path to the target and follow it. Calling it every frame keeps the path up to date while the player moves; for a few enemies that's cheap.
- **A target off the NavMesh**, such as a wander spot in the pond, sends the agent to the nearest point it can reach.
- **`StopMoving`** stops the agent and forgets its path. `ChangeState` calls it, where the old version set `moveDirection` to zero.

That's all. The states, their rules and `ChangeState`'s jobs are untouched: the brain didn't change, only the legs. Because deciding and moving were kept apart in T06, swapping straight lines for pathfinding only touched a few lines.

Add it: select **Slime 1** and **Slime 2** and add **Pathfinding Enemy AI**. Set **Give Up Range** to **10** and leave everything else as it is. (The ranges measure a straight line, but the way out of the pen is longer: with 7, the pen slime gave up halfway out of the gate.)

Test it: press Play and walk past the pen: **Slime 2** heads for the gate, comes out and chases you. Stand below the pond: **Slime 1** runs round the end of it instead of getting stuck. Run away and they walk home the same way.

Check your work

- ☐ The **NavMesh** object has a Navigation Surface and a Navigation CollectSources2d (**Override By Grid** ticked), rotated −90 on X.
- ☐ **Ground**, **Water** and **Fences** each have a Navigation Modifier, with Water and Fences **Not Walkable**.
- ☐ After baking, the blue NavMesh has holes round the water and fences, with a gap through the gate.
- ☐ Both slimes have a **Nav Mesh Agent** (Radius 0.4, Acceleration 20) and a **Kinematic** Rigidbody 2D.
- ☐ The slimes still idle, wander, chase, attack and go home as in T06b.
- ☐ The pen slime leaves through the gate and goes back in the same way.
- ☐ The pond slime goes round the pond, never into the water's edge.
- ☐ The slimes stay upright and don't turn to face where they walk.
- ☐ Compare with `CRE132_T07_Pathfinding_Finished` if something behaves differently.

Key points

- **A NavMesh is a map of where an agent can stand.** The agent plans a path across it and follows it, round anything that isn't on the map.
- **Keeping deciding (the state machine) apart from moving (the agent)** meant swapping straight lines for pathfinding changed only the movement code.
- **The NavMesh is baked in the editor.** If you change the walls, bake again.

If something goes wrong

- **"The type or namespace name 'NavMeshAgent' could not be found":** the `using UnityEngine.AI;` line is missing.
- **"The name 'body' (or 'moveSpeed') does not exist":** a line still uses the old fields. Double-click the error to find it.
- **Unity won't add the script ("the script class cannot be found"):** the class line still says `EnemyAI`; it must say `PathfindingEnemyAI`.
- **There's no Navigation Surface in Add Component:** the NavMeshPlus package hasn't finished downloading. Check **Window > Package Manager**, then try again.
- **A yellow warning says the NavMeshSurface is not rotated:** click **Rotate Surface to XY** on CollectSources2d, then bake again.

- **No blue appears after baking: NavMesh** isn't selected in the Hierarchy, or **Ground** has no Navigation Modifier, or **Override By Grid** is unticked. Fix it and bake again.
- **"SetDestination can only be called on an active agent that has been placed on a NavMesh":** the NavMesh hasn't been baked, or the slime starts off it. Select **NavMesh** and click **Bake**.
- **The slime seems to vanish or turns edge-on when you press Play:** the `updateRotation` and `updateUpAxis` lines are missing from `Awake`.
- **The slime walks through the fences, or into the pond: Fences** (or **Water**) needs a Navigation Modifier set to **Not Walkable**, and you need to bake again.
- **The slime jitters or gets pushed about:** its **Rigidbody 2D** is still **Dynamic**. Set it to **Kinematic**.

Try it

1. Paint more fence tiles with the **MysticWoods** palette, bake again, and watch the slimes find new routes.
2. Make the pen's gate narrower than the agent and see the slime give up on getting out.
3. Make the slime chase only if `agent.CalculatePath` finds a complete path (`NavMeshPathStatus.PathComplete`).
4. Add a third area, *Mud*, with a higher cost, and see paths go round it when they can.